From the fork to NGSolve master: what to merge, and why

Edoardo Bonetti November 9, 2026

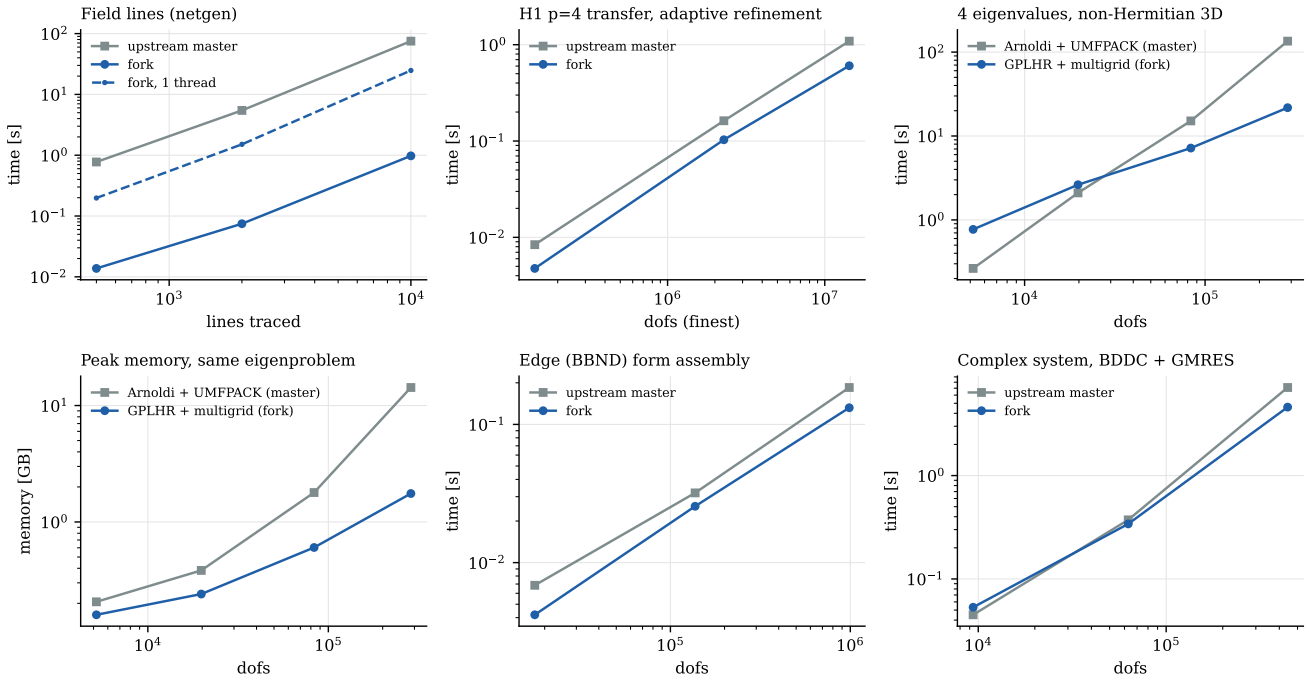
In short. The fork (`EdoardoBonetti/ngsolve`, branch `ngsolve`, merged with master `6e701b568` on 9 November) holds **12 fixes for bugs in current master**, each with a regression test that fails on master, and **six changes measured at three problem sizes**: four are faster at every size, two from medium size on. Each change is a separate small merge request of cherry-picked commits; all but three apply to master without conflicts.

1 Speed: upstream master against the fork, small / medium / large

Both builds compiled with identical options (Release, UMFPACK, CPU only) on MUSICA; all runs on one AMD Zen 4 node (48 cores, 192 GB); each upstream/fork pair runs back to back in the same job. Times are wall times of the operation itself; every fork result agrees with upstream (matrix norms, eigenvalues, traced length, interpolation errors).

	small	**medium**	**large**
Field lines	0.774 → 0.0137 s, **56**×	5.45 → 0.0747 s, **73**×	75.2 → 0.976 s, **77**×
H1 transfer (`localhoprolongation`)	0.00839 → 0.00476 s, **1.8**×	0.162 → 0.103 s, **1.6**×	1.09 → 0.605 s, **1.8**×
Eigenvalues (GPLHR + MG vs Arnoldi + UMFPACK)	2.1 → 2.62 s, **0.80**×	15.1 → 7.18 s, **2.1**×	135 → 21.7 s, **6.2**×
SIMD edge forms	0.00686 → 0.00419 s, **1.6**×	0.032 → 0.0256 s, **1.2**×	0.185 → 0.132 s, **1.4**×
Complex system: real BDDC vs best complex BDDC	0.045 → 0.0532 s, **0.85**×	0.373 → 0.341 s, **1.09**×	7.11 → 4.59 s, **1.5**×
`mesh.Refine`	1.01 → 0.963 s, **1.05**×	11.1 → 10.6 s, **1.05**×	121 → 116 s, **1.05**×

Sizes (small/medium/large): field lines 500/2 000/10 000 lines on 6 k/47 k/379 k tets; H1 $p = 4$ transfer to 0.14/2.3/14.3 M dofs; eigenproblem H1 $p = 2$ with 20/83/287 k dofs; edge form VectorH1 $p = 3$ with 18/137/987 k dofs; complex system H1 $p = 3$ with 9/63/443 k dofs; `Refine` to 3.2/20/154 M tets.



- **Field lines** (netgen `FieldLineCalc`): testing the previous element before the search tree gives 3–4× on one thread; the lines are independent and now run in parallel (25× more on 48 threads). Same traced length; 3.6 % more segments, because master carries the RK step size from one line into the next.
- **H1 transfer**: cost proportional to the refined elements; memory 2.4 → 1.3 GB at 14.3 M dofs; equal to `hoprolongation` to 10^{-14}.
- **Eigenvalues** of $-\Delta u + b \cdot \nabla u = \lambda u$ (unit cube, $b = (8, 6, 4)$): both return 58.61 and the triple 88.22. GPLHR needs only products with A, M and a preconditioner; with geometric multigrid it wins from about 30 k dofs and needs less memory at every size (14.6 → 1.8 GB at 287 k). Below that, a direct solver is hard to beat (5 k dofs: 0.26 vs 0.77 s). At $p = 2$, BDDC is no help: its wirebasket is the whole space.
- **Complex systems**: master cannot apply a real preconditioner to a complex vector (Section 2), so it needs a complex BDDC. The fork's real BDDC: same iterations (16/17/19), 28 % less memory and 1.5× faster at 443 k dofs; equal on small problems (0.05 s).
- **SIMD edge forms**: identical matrices. `mesh.Refine`: 4–5 % at every size, a side benefit.

2 Bugs in current master, fixed in the fork (one test each)

Where	Symptom on master `6e701b568`	failing tests
`ComputeNormalAndMeasureET`	SIMD edge tangent is 0 (since the 3 Nov rework): BBND forms with `specialcf.tangential` assemble to zero	1/3
`EdgeFaceTangentialVectors`	no SIMD evaluation: the integrator falls back to scalar code for good	1/2
`u.Other(bnd=g)`	boundary-facet term missing from the assembled matrix	2/2
`IntegrationRuleSpace.Set`	order ≥ 6 in 3D: SIMD blocks of 64 points land on the wrong dofs (garbage, NaN)	6/6
`Compile(...)`	with `code_uses_equivalence_keys`: the generated C++ does not compile	1/1
periodic `.Other()`	evaluated at wrong points when the identification reorders vertex numbers	4/8
periodic `Set(dual=True)`	cached duality inverse picked from unrenumbered vertices (sent as GitHub PR #102)	8/10
L2 `hoprolongation`	spaces created after `Refine`: L2 error $10^{-5} \to 9 \cdot 10^{-2}$	8/15
`MultiVector.Orthogonalize()`	without `ipmat`: not orthonormal from 4 vectors on (block coupling used transposed)	30/64
`DeviceCG`, `DeviceTFQMR`	NaN after exact convergence between residual checks	5/7
real BDDC × complex vector	raises "illegal combination of scalar types": no real preconditioner for complex systems	1/6
netgen `SplitAlfeld`	3D split vertex outside the tet: inverted elements	1/3

3 MPI: what carries over, pros and cons

Assessment from the code; none of it is measured with MPI yet.

Change	With MPI-distributed meshes
bug fixes, SIMD edge forms	element-local or vector-local: work unchanged on every rank.
real operator on complex vector	written through `BaseMatrix` with the operator's own real `ParallelDofs` (`parallelvvector.cpp` is covered); needs an `np` ≥ 2 test.
GPLHR	needs only products with A, M, a preconditioner and inner products, so it should run on `ParallelMatrix` without changes. The preconditioner changes: BDDC ($p \geq 3$) or hypre BoomerAMG, since geometric multigrid is shared-memory only. Arnoldi with shift-invert would need MUMPS, whose memory grows with the fill-in.
`localhoprolongation`	per-element formulas; needs the parent relation on each rank after parallel refinement.
field lines	thread-parallel only; lines cross subdomain borders, so they stay on one rank.

Pros of going MPI.

- Memory: Arnoldi + UMFPACK already needs 14.6 GB at 287 k dofs, and a direct solver's memory grows faster than the problem. With GPLHR and a distributed preconditioner, the memory is spread over the nodes.
- GPLHR is the eigensolver that fits MPI: no global factorization, the same Python code on every rank.
- The element-local fixes need no extra work, so MPI users get them for free.

Cons.

- On one node, threads already scale (field lines 25× from 1 to 48 threads at 10 000 lines); MPI pays off only beyond one node's memory.
- Build and test cost: MPI build with `mpi4py` and MUMPS or hypre, and an `np` ≥ 2 test in the CI for every change.
- Adaptive refinement and geometric multigrid on distributed meshes are the least mature parts (load balance after refinement), so the `localhoprolongation` and multigrid gains do not transfer directly.

4 Proposed merge requests (GitLab, cherry-picked, smallest first)

1. The bug fixes of Section 2, one MR each with its test (no API change).
2. SIMD edge forms: `EdgeFaceTangentialVectors` SIMD + tangential fix (needs a small rebase on the 3 Nov rework).
3. `H1(localhoprolongation=True)`, and prolongations for HCurlCurl and refined L2 spaces.
4. Real operators on complex vectors (`BaseMatrix::MultComplexByParts`; small rebase).
5. `solvers.GPLHR`: new functions in `eigenvalues.py` (Python with scipy, like LOBPCG; existing code unchanged).
6. netgen: parallel field lines with the element hint, `Refine` cluster-update skip, 1D bisection.

Everything reruns with one command each: `upstream-candidates/musica/submit_sweep.sh` (this sweep) and `upstream-candidates/run_ab.sh` (the tests of Section 2, run on a Mac); raw results with cost and provenance in `upstream-candidates/results/`.